SQL Lab-20

Implement Window Functions for Advanced Data Analysis

From the table STUDENT perform the following queries:

Part – A

1. Display rank of students based on SPI.

```
SELECT
    SNAME,
    SPI,
    RANK() OVER (ORDER BY SPI DESC) AS StudentRank
FROM STUDENT
```

2. Display dense rank of students based on SPI.

```
SELECT
    SNAME,
    SPI,
    DENSE_RANK() OVER (ORDER BY SPI DESC) AS DenseRank
FROM STUDENT
```

3. Display sequential number for each student record.

```
SELECT
    SNAME,
    SPI,
    ROW_NUMBER() OVER (ORDER BY STDID) AS SerialNo
FROM STUDEN
```

4. Display branch-wise rank of students.

```
SELECT
    SNAME,
    BRANCH,
    SPI,
    RANK () OVER
    (
        PARTITION BY BRANCH
        ORDER BY SPI DESC
    ) AS BranchRank
FROM STUDENT
```

5. Display branch-wise dense ranking of students.

```
SELECT
    SNAME,
    BRANCH,
    SPI,
    DENSE_RANK() OVER
    (
        PARTITION BY BRANCH
        ORDER BY SPI DESC
    ) AS BranchDenseRank
FROM STUDENT
```

6. Display branch-wise sequential numbering of students.

```
SELECT
    SNAME,
    BRANCH,
    SPI,
    ROW_NUMBER() OVER
    (
        PARTITION BY BRANCH
        ORDER BY SPI DESC
    ) AS BranchSerialNo
FROM STUDENT
```

7. Display SNAME, Current SPI, Previous SPI and SPI Difference with previous student in ascending order of SPI.

```
SELECT
    SNAME,
    SPI AS CurrentSPI,
    LAG(SPI) OVER (ORDER BY SPI ASC) AS Previous_SPI,
    SPI - LAG(SPI) OVER (ORDER BY SPI ASC) AS SPI_Difference
FROM STUDENT
ORDER BY SPI ASC
```

8. Display SNAME, Current SPI, Next SPI and SPI Difference with next student in descending order of SPI.

```
SELECT
    SNAME,
    SPI AS CurrentSPI,
    LEAD(SPI) OVER (ORDER BY SPI DESC) AS Next_SPI,
    SPI - LEAD(SPI) OVER (ORDER BY SPI DESC) AS SPI_Difference
FROM STUDENT
ORDER BY SPI DESC
```

9. Display top 3 students based on SPI.

```
SELECT * FROM

(
    SELECT
        SNAME,
        SPI,
        RANK () OVER (ORDER BY SPI DESC) AS Rnk
    FROM STUDENT
)
WHERE Rnk <= 3
```

10. Display top 2 students from each branch.

```
SELECT * FROM
(
    SELECT
        SNAME,
        BRANCH,
        SPI,
        ROW_NUMBER() OVER
        (
            PARTITION BY BRANCH
            ORDER BY SPI DESC
        ) AS Rn
    FROM STUDENT
)
WHERE Rn<=2
```

Part – B

11. Display 5th highest SPI.

```
SELECT * FROM
(
    SELECT
        SPI,
        DENSE_RANK() OVER (ORDER BY SPI DESC) AS Rnk
    FROM STUDENT
)
WHERE Rnk=5
```

12. Display 6th highest SPI.

```
SELECT * FROM
(
    SELECT
        SPI,
        DENSE_RANK() OVER (ORDER BY SPI DESC) AS Rnk
    FROM STUDENT
)
WHERE Rnk=6
```

13. Display students having same ranking.

```
SELECT SNAME, SPI, Rnk

FROM

(

       SELECT
           SNAME,
           SPI,
           RANK() OVER (ORDER BY SPI DESC) AS Rnk
       FROM STUDENT

) AS DATA1

WHERE Rnk IN

(

    SELECT Rnk

    FROM

    (

        SELECT RANK() OVER (ORDER BY SPI DESC) AS Rnk

        FROM STUDENT

    ) AS DATA2

    GROUP BY Rnk

    HAVING COUNT(*) > 1

)
```

14. Display SNAME, Previous SPI, Current SPI and Next SPI based on ascending order of SPI.

```
SELECT
    SNAME,
    LAG(SPI) OVER (ORDER BY SPI ASC) AS Previous_SPI,
    SPI AS Current_SPI,
    LEAD(SPI) OVER (ORDER BY SPI ASC) AS Next_SPI
FROM STUDENT
ORDER BY SPI ASC;
```

15. Display topper of each branch.

```
SELECT * FROM
(
    SELECT
        SNAME,
        BRANCH,
        SPI,
        ROW_NUMBER() OVER
        (
            PARTITION BY BRANCH
            ORDER BY SPI DESC
        ) AS Rn
    FROM STUDENT
)
WHERE Rn = 1;
```

Part – C

16. Display students whose SPI is greater than the previous student and less than the next student.

```
SELECT * FROM
(
    SELECT
        SNAME,
        SPI,
        LAG(SPI) OVER (ORDER BY SPI ASC) AS Previous_SPI,
        LEAD(SPI) OVER (ORDER BY SPI ASC) AS NextSPI
    FROM STUDENT
)
WHERE SPI > Previous_SPI
AND SPI < Next_SPI;
```

17. Display branch-wise second topper students.

```
SELECT * FROM
(
    SELECT
        SNAME,
        BRANCH,
        SPI,
        ROW_NUMBER() OVER
        (
            PARTITION BY BRANCH
            ORDER BY SPI DESC
        ) AS Rn
    FROM STUDENT
)
WHERE Rn = 2;
```

OR

```
SELECT * FROM
(
    SELECT
        SNAME,
        BRANCH,
        SPI,
        DENSE_RANK() OVER
        (
            PARTITION BY BRANCH
            ORDER BY SPI DESC
        ) AS Rnk
    FROM STUDENT
)

WHERE Rnk = 2;
```

18. Display students whose rank and dense rank are different.

```
SELECT * FROM
(
    SELECT
        SNAME,
        SPI,
        RANK() OVER (ORDER BY SPI DESC) AS Rnk,
        DENSE_RANK() OVER (ORDER BY SPI DESC) AS DenseRnk
    FROM STUDENT
)
WHERE Rnk != DenseRnk;
```

OR

```
SELECT * FROM
(
    SELECT
        SNAME,
        SPI,
        RANK() OVER (ORDER BY SPI DESC) AS Rnk,
        DENSE_RANK() OVER (ORDER BY SPI DESC) AS DenseRnk
    FROM STUDENT
)
WHERE Rnk <> DenseRnk;
```

19. Display consecutive students having same branch ordered by SPI.

```
SELECT * FROM
(
    SELECT
        SNAME,
        BRANCH,
        SPI,
        LAG(BRANCH) OVER (ORDER BY SPI ASC) AS PreviousBranch
    FROM STUDENT
)
WHERE BRANCH = PreviousBranch
ORDER BY SPI ASC;
```

20. Display students whose SPI difference with previous student is maximum.

```
SELECT SNAME,SPI,Difference

FROM
(
    SELECT
        SNAME,
        SPI,
        SPI-LAG(SPI) OVER (ORDER BY SPI ASC) AS Difference
    FROM STUDENT
) AS DATA1
WHERE Difference=

(

    SELECT MAX(Difference)

    FROM

    (

        SELECT
            SPI-LAG(SPI) OVER (ORDER BY SPI ASC) AS Difference
        FROM STUDENT

    ) AS DATA2

)
```